

ARQUITETURA DE SOFTWARE

O QUE É?

A arquitetura de software permite que, no desenvolvimento de projetos, seja feito o melhor uso possível do tempo e do código. Todos os projetos de software atuais seguem algum tipo de arquitetura que comportam as suas necessidades.

Conforme definição da norma ISO/IEC/IEEE 42.010:2022, “arquitetura de software é a estrutura fundamental de um sistema, composta por suas partes, os relacionamentos entre essas partes e o relacionamento com o ambiente, projetada para atender aos requisitos do sistema”.

POR QUE SURTIU?

Na década de 1960, ocorreu a crise do software, quando os sistemas passaram a ficar mais complexos e mais difíceis de serem feitos. Em vez de consertar alguma funcionalidade, era mais fácil começar um projeto do zero.

Conforme os projetos de software foram ficando maiores, mais difíceis – e, conseqüentemente, mais caros – de manter, houve a necessidade de criar formas de organizar o código de fonte de uma maneira mais sustentável.

O conceito de arquitetura de software começou a ganhar forma entre as décadas de 1960 e 1970, mas foi em 1980 que ele se consolidou como uma disciplina fundamental da engenharia de software.

À época, tratava-se de uma forma de se projetar um código com uma certa lógica, de modo que fosse mais fácil realizar a sua manutenção.



5m

ARQUITETURA MONOLÍTICA

As questões de concurso não trazem questões específicas sobre a arquitetura monolítica, porque ela foi a primeira, então era bem primitiva. Contudo, ela é usada para ser comparada com outras arquiteturas.

A arquitetura monolítica surgiu em 1970 e foi consolidada em 1980. Trata-se da primeira abordagem estruturada para desenvolvimento de sistemas.

Seu objetivo é simplificar o desenvolvimento e implementação de sistemas de software. Todas as funcionalidades do sistema (interface de usuário, lógica de negócios, acesso a dados) são desenvolvidas juntas, de forma interdependente e em um único arquivo executável.

ARQUITETURA CLIENTE-SERVIDOR

A arquitetura cliente-servidor foi criada em 1980, com o objetivo de distribuir a carga de trabalho para gerir os recursos de forma mais eficiente.

O cliente interage com o usuário e envia solicitações ao servidor, ao passo que o servidor processa as solicitações, executa lógica de negócios e retorna resultados ao cliente.

ARQUITETURA ORIENTADA A OBJETOS

A arquitetura orientada a objetos também surgiu em 1980. Seu objetivo é organizar o desenvolvimento de software de uma forma que imita o mundo real e facilitar a modularidade, reutilização e manutenção do código.

Em relação ao funcionamento, ela organiza o sistema em objetos, que são instâncias de classes. As classes são como um formulário, ao passo que os objetos são como o preenchimento do formulário.

O objeto é o elemento real, a classe em si é apenas o molde que se deve seguir para criar algo de verdade.



10m

ARQUITETURA MVC

Essa arquitetura surgiu em 1970 e foi consolidada em 1990. Seu objetivo é dividir as aplicações em três componentes por uma organização mais clara.

O “model” representa os dados da aplicação e a lógica de negócios. O “view” apresenta os dados ao usuário e o “controller”, que é intermediário entre “model” e “view”, recebe as entradas do usuário, processa, e determina qual view será exibida.

ARQUITETURA EM CAMADAS

A arquitetura em camadas surgiu em 1980 e foi consolidada em 1990. O objetivo é organizar o software em camadas de forma que cada uma tenha um objetivo específico.

Ela possui três camadas principais, podendo ser estendida conforme a necessidade. A apresentação é responsável pela interface do usuário e interação com usuário final (páginas web).

Os negócios processam entradas e aplicam regras de negócios (serviços, validações) e os dados gerenciam a persistência de dados (repositórios, conexão com banco de dados).

Exemplo: Em um cadastro de usuário, a camada de apresentação seria o preenchimento de e-mail e senha, a verificação seria a confirmação de que o usuário existe e os dados seriam a persistência dos dados recebidos.

ARQUITETURA DE OBJETOS DISTRIBUÍDOS

Essa arquitetura surgiu em 1980 e foi consolidada em 1990. O objetivo é permitir que objetos de software interajam como se estivessem no mesmo ambiente, mesmo que estejam em sistemas diferentes.

Há objetos que residem em diferentes máquinas conectadas por uma rede. Eles interagem entre si através de protocolos de comunicação e middleware específico (CORBA, RMI, DCOM).



PADRÕES DE COMUNICAÇÃO

Os padrões de comunicação não são uma arquitetura, mas uma forma de se fazer uma transferência de dados. O SOAP surgiu em 1990 com o objetivo de permitir comunicação entre aplicações distribuídas em diferentes plataformas, através de XML.

ARQUITETURA PEER-TO-PEER

Surgiu em 1990, consolidando-se em 2000. Seu objetivo é distribuir tarefas e recursos de forma descentralizada entre os participantes da rede (peers), sem a necessidade de um servidor central.

Todos os nós (peers) na rede são iguais e podem atuar simultaneamente como clientes e servidores.

ARQUITETURA ORIENTADA A SERVIÇOS (SOA)

Surgiu em 2000, com o objetivo de permitir que diferentes serviços de software (distribuídos em diferentes sistemas e plataformas) interajam e funcionem juntos.

A aplicação é composta por serviços distintos. Eles se comunicam por meio de interfaces bem definidas, fazendo uso de protocolos de comunicação, como HTTP.

PADRÕES DE COMUNICAÇÃO

O REST veio no ano 2000. Seu objetivo é criar APIs web simples e escaláveis e ele funciona com requisições HTTP (get, post, put, delete).

ARQUITETURA EVENT-DRIVEN

Essa arquitetura surgiu no ano 2000, para criar sistemas que reagem rapidamente a eventos e mudanças de estado. Aqui, o fluxo de trabalho é determinado por eventos – que podem ser qualquer ocorrência significativa detectada e tratada pelo sistema.

ARQUITETURA HEXAGONAL

A arquitetura hexagonal surgiu em 2000, para criar um design de software que seja independente de tecnologia e frameworks. O núcleo corresponde à lógica de negócio e às regras de domínio.

Os ports (ou portas) são as interfaces que definem a interação do núcleo com o exterior. Os adapters (ou adaptadores) são implementações das portas, que adaptam a interface externa para a interface esperada pelo núcleo da aplicação e vice-versa.



20m

ARQUITETURA DE MICROSERVIÇOS

Essa arquitetura surgiu em 2010, com o objetivo de dividir uma aplicação monolítica em serviços menores, independentes e focados em funcionalidades específicas.

Nessa arquitetura, cada serviço é responsável por uma funcionalidade específica e se comunica com outros serviços através de APIs. Ela costuma ser bastante cobrada em contraposição com a arquitetura monolítica.

ARQUITETURA SERVERLESS

Essa arquitetura também surgiu em 2010, mas com o objetivo de permitir que os desenvolvedores focassem exclusivamente na lógica de negócios, ao passo que a nuvem gerencia a escalabilidade, disponibilidade e manutenção dos servidores.

A execução do código é totalmente gerenciada por um provedor de nuvem.

Este material foi elaborado pela equipe pedagógica do Gran Concursos, de acordo com a aula preparada e ministrada pela professora Ana Júlia Batista de Souza.

A presente gravação tem como objetivo auxiliar no acompanhamento e na revisão do conteúdo ministrado na videoaula. Não recomendamos a substituição do estudo em vídeo pela leitura exclusiva deste material.
